

Research Collaboration Portal - Use Cases and SQL Queries

This document describes the main use cases supported by the application and the SQL queries executed by the backend routes. The goal is to show how the user interface maps to backend API routes and database operations.

All dynamic values are passed through mysql2 parameter placeholders (?) instead of direct string concatenation. This reduces SQL injection risk. Passwords are hashed using PBKDF2 before storage.

Student Registration

A new student creates an account with name, email, password, and major.

API route: POST /api/register

Executed query/queries:

```
select * from Student where semail = ?  
insert into Student(sname, semail, spassword, major) values (?, ?, ?, ?)
```

Professor Registration

A new professor creates an account with name, email, password, and department.

API route: POST /api/register

Executed query/queries:

```
select * from Professor where pemail = ?  
insert into Professor(pname, pemail, ppassword, department) values (?, ?, ?, ?)
```

Student Login

A student logs in using email and password. The backend verifies the password against the stored hash and returns student user data.

API route: POST /api/login

Executed query/queries:

```
select * from Student where semail = ?  
update Student set spassword = ? where Student_ID = ? -- only when upgrading an old plaintext password
```

Professor Login

A professor logs in using email and password. The backend verifies the password against the stored hash and returns professor user data.

API route: POST /api/login

Executed query/queries:

```
select * from Professor where pemail = ?  
update Professor set ppassword = ? where Prof_ID = ? -- only when upgrading an old plaintext password
```

Professor Views Own Projects

A professor opens View Projects. The backend returns only projects supervised by that professor, with assigned student names and roles.

API route: GET /api/professors/:profId/projects

Executed query/queries:

```
select Project.Proj_ID, Project.projName, Project.ptitle, Project.pstatus, Project.pdescription,
group_concat(Student.sname separator ', ') as studentNames,
group_concat(ProjectMember.role separator ', ') as studentRoles
from Project
left join ProjectMember on Project.Proj_ID = ProjectMember.Proj_ID
left join Student on ProjectMember.Student_ID = Student.Student_ID
where Project.Prof_ID = ?
group by Project.Proj_ID, Project.projName, Project.ptitle, Project.pstatus, Project.pdescription
```

Create Project

A professor creates a new research project with name, title, status, description, and professor id.

API route: POST /api/projects

Executed query/queries:

```
insert into Project(projName, ptitle, pstatus, pdescription, Prof_ID) values (?, ?, ?, ?, ?)
```

Update Project Status

A professor updates an existing project status to Planned, Active, or Completed.

API route: PUT /api/projects/:projectId/status

Executed query/queries:

```
update Project set pstatus = ? where Proj_ID = ?
```

Add Student to Project

A professor assigns a student to a project and gives the student a project role.

API route: POST /api/project-members

Executed query/queries:

```
insert into ProjectMember(Student_ID, Proj_ID, role, joinDate) values (?, ?, ?, CURDATE())
```

Student Views Assigned Projects

A student opens View Projects. The backend returns only projects where the student appears in ProjectMember, plus teammate information for the right sidebar.

API route: GET /api/students/:studentId/projects

Executed query/queries:

```
select Project.Proj_ID, Project.projName, Project.ptitle, Project.pstatus, Project.pdescription,
ProjectMember.role, ProjectMember.joinDate,
(select group_concat(concat(Student.sname, ' :: ', ProjectMember.role) separator '||')
from ProjectMember join Student on ProjectMember.Student_ID = Student.Student_ID
where ProjectMember.Proj_ID = Project.Proj_ID and ProjectMember.Student_ID <> ?) as teammates
from ProjectMember
join Project on ProjectMember.Proj_ID = Project.Proj_ID
where ProjectMember.Student_ID = ?
```

Create Milestone

A professor creates a milestone attached to a project. The frontend uses a date input so the backend receives a MySQL-compatible due date.

API route: POST /api/milestones

Executed query/queries:

```
insert into Milestone(mtitle, mduedate, mstatus, mdescription, Proj_ID) values (?, ?, ?, ?, ?)
```

View Project Milestones

A student or professor views milestone details for a selected project.

API route: GET /api/projects/:projectId/milestones

Executed query/queries:

```
select Mstone_ID, mtitle, mduedate, mstatus, mdescription from Milestone where Proj_ID = ?
```

Update Milestone

A professor updates an existing milestone title, due date, status, description, or project association.

API route: PUT /api/milestones/:milestoneId

Executed query/queries:

```
update Milestone set mtitle = ?, mduedate = ?, mstatus = ?, mdescription = ?, Proj_ID = ? where Mstone_ID = ?
```

Submit Progress Report

A student submits a progress report for a selected milestone. The report includes title, text, optional file path/link, milestone id, and student id.

API route: POST /api/progress-reports

Executed query/queries:

```
insert into ProgressReport(prtitle, prtext, createdAt, filepath, Mstone_ID, Student_ID) values (?, ?, CURDATE(), ?, ?, ?)
```

Student Views Own Progress Reports

A student views their submitted reports, including project, milestone, file link, and any professor feedback.

API route: GET /api/students/:studentId/progress-reports

Executed query/queries:

```
select ProgressReport.PR_ID, ProgressReport.prtitle, ProgressReport.prtext, ProgressReport.createdAt,
ProgressReport.filepath, Milestone.mtitle, Project.projName,
Feedback.fdbckTitle, Feedback.fdbckText, Feedback.createdAt as feedbackCreatedAt
from ProgressReport
join Milestone on ProgressReport.Mstone_ID = Milestone.Mstone_ID
join Project on Milestone.Proj_ID = Project.Proj_ID
left join Feedback on ProgressReport.PR_ID = Feedback.PR_ID
where ProgressReport.Student_ID = ?
order by ProgressReport.createdAt desc
```

Professor Views Student Reports

A professor views reports submitted for milestones under projects they supervise. This prevents professors from seeing unrelated project reports.

API route: GET /api/professors/:profId/progress-reports

Executed query/queries:

```
select ProgressReport.PR_ID, ProgressReport.prtitle, ProgressReport.prtext, ProgressReport.createdAt,
ProgressReport.filepath, Student.sname as studentName, Milestone.mtitle, Project.projName,
Feedback.fdbckTitle, Feedback.fdbckText, Feedback.createdAt as feedbackCreatedAt
from ProgressReport
join Student on ProgressReport.Student_ID = Student.Student_ID
join Milestone on ProgressReport.Mstone_ID = Milestone.Mstone_ID
join Project on Milestone.Proj_ID = Project.Proj_ID
left join Feedback on ProgressReport.PR_ID = Feedback.PR_ID
where Project.Prof_ID = ?
order by ProgressReport.createdAt desc
```

Add or Update Feedback

A professor adds feedback to a progress report. If feedback already exists for the same report, it is updated.

API route: POST /api/feedback

Executed query/queries:

```
insert into Feedback(fdbckTitle, fdbckText, createdAt, Prof_ID, PR_ID)
values (?, ?, CURDATE(), ?, ?)
on duplicate key update
fdbckTitle = values(fdbckTitle),
fdbckText = values(fdbckText),
createdAt = CURDATE(),
Prof_ID = values(Prof_ID)
```

Security and Filtering Summary

Authentication supports two roles: Student and Professor. The backend checks the role selected by the user and validates credentials against the corresponding table.

Password hashing is handled in the backend using Node crypto PBKDF2. New passwords are stored as pbkdf2\$salt\$hash instead of plaintext. During login, the entered password is hashed using the stored salt and compared securely.

Student project access is filtered by ProjectMember.Student_ID. Professor project and report access is filtered by Project.Prof_ID. This keeps each role from seeing unrelated project data.

The frontend stores the current user in localStorage so refreshing the browser does not immediately log the user out. This is suitable for the local course demo; a deployed production app would normally use sessions or JWT tokens.